

Cahier des Charges Complet

Projet : Automatisation DevOps de bout en bout pour un Chatbot IA évolutif (ChatOps AI)

1. Présentation Générale du Projet

1.1 Contexte

Les entreprises modernes ont besoin de plateformes intelligentes capables d'automatiser les échanges utilisateurs, de centraliser les opérations IT et de garantir des déploiements rapides et fiables.

Le projet **ChatOps AI** vise à concevoir une plateforme DevOps complète permettant :

- Le déploiement automatisé d'un chatbot IA ;
- L'intégration continue (CI) ;
- Le déploiement continu (CD) ;
- L'orchestration cloud-native ;
- Le monitoring temps réel ;
- La scalabilité automatique ;
- La gestion de l'infrastructure via Infrastructure as Code (IaC).

Le projet repose sur une architecture moderne utilisant FastAPI, Docker, Kubernetes, Terraform, AWS, GitHub Actions, GitLab CI et Jenkins.

2. Objectifs du Projet

2.1 Objectifs Principaux

Objectifs fonctionnels

- Développer une API de chatbot IA robuste.
- Permettre une communication rapide via HTTP REST.
- Garantir une haute disponibilité.
- Fournir des endpoints de supervision.

Objectifs DevOps

- Automatiser le cycle de vie logiciel.
- Réduire les erreurs humaines.
- Garantir des déploiements rapides.
- Assurer la traçabilité des versions.
- Mettre en place des tests automatisés.
- Superviser les performances applicatives.

Objectifs Cloud

- Déployer automatiquement l'infrastructure AWS.
 - Permettre une montée en charge dynamique.
 - Réduire les coûts d'exploitation.
-

3. Périmètre du Projet

Inclus dans le projet

Backend

- API FastAPI
- Endpoints REST
- Endpoint healthcheck
- Endpoint métriques Prometheus

DevOps

- Dockerisation
- Kubernetes
- CI/CD GitHub Actions
- CI/CD GitLab
- Jenkins Pipeline
- Terraform AWS

Monitoring

- Prometheus
- Grafana
- Collecte des métriques
- Dashboard temps réel

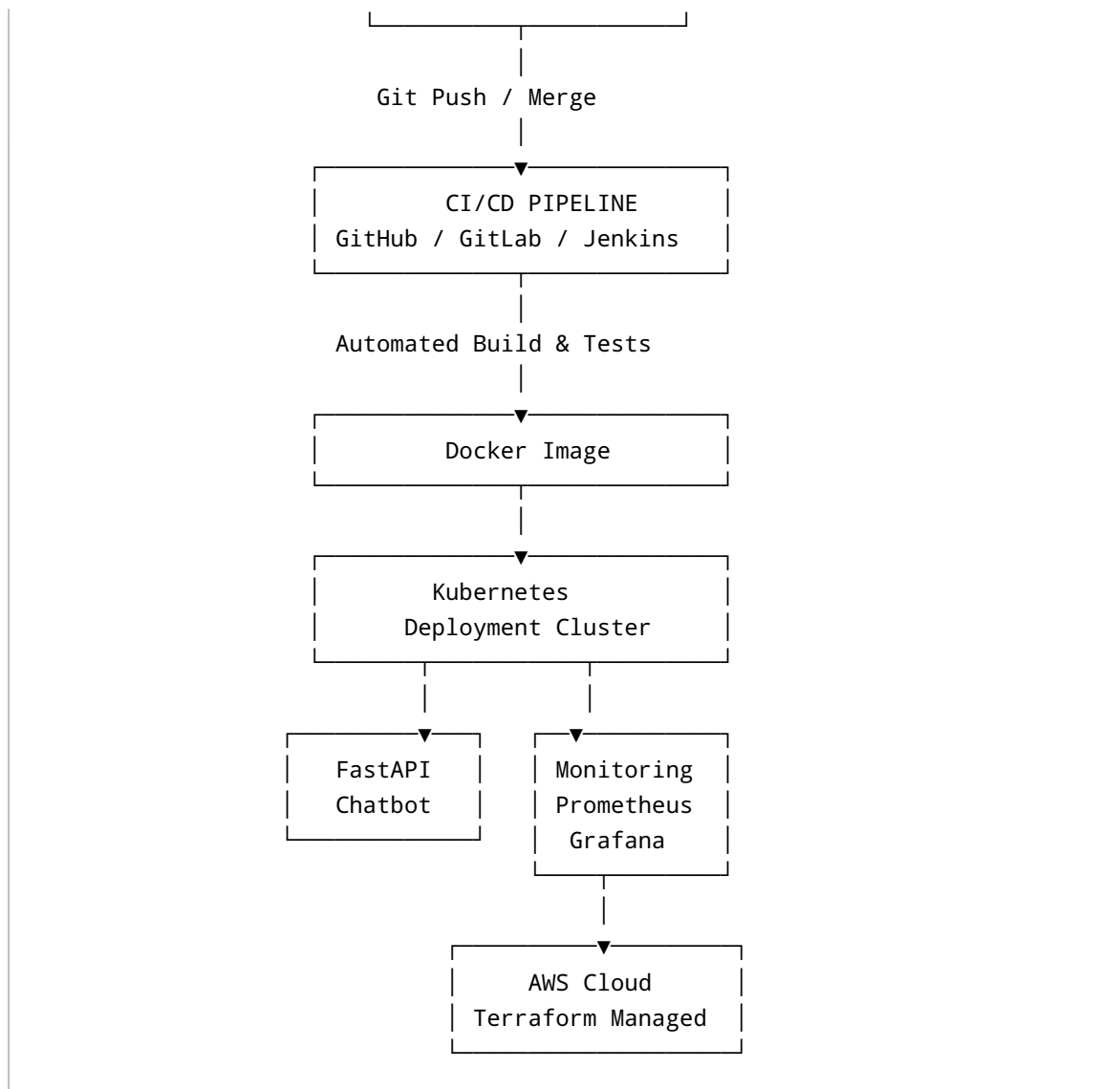
Tests

- Tests unitaires
 - Tests API
 - Linting
 - Validation CI
-

4. Architecture Fonctionnelle

4.1 Vue globale de l'architecture





5. Description Technique des Composants

5.1 FastAPI

Description

FastAPI est utilisé comme framework backend principal.

Rôle

- Gestion des requêtes API
- Communication chatbot
- Exposition des endpoints REST
- Endpoint de monitoring

Avantages

- Haute performance
- Documentation Swagger automatique
- Compatible async
- Facilité de maintenance

Endpoints prévus

Endpoint	Description
/	Accueil API
/health	Vérification état service
/metrics	Métriques Prometheus
/chat	Endpoint chatbot IA

5.2 Docker

Description

Docker permet de conteneuriser l'application.

Objectifs

- Portabilité
- Isolation
- Standardisation des environnements
- Déploiement reproductible

Fonctionnement

Application → Docker Image → Container Runtime

Livrables Docker

- Dockerfile
 - Docker Compose
 - Registry push
-

5.3 Kubernetes

Description

Kubernetes orchestre les conteneurs Docker.

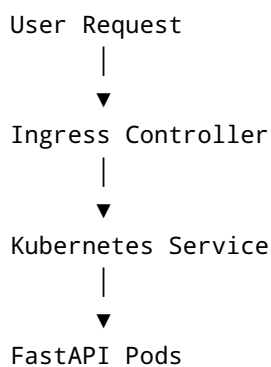
Fonctions principales

- Déploiement automatique
- Load balancing
- Scalabilité
- Self-healing
- Rolling updates

Fichiers Kubernetes

Fichier	Rôle
deployment.yaml	Déploiement des pods
service.yaml	Exposition réseau
ingress.yaml	Routage externe

Architecture Kubernetes



5.4 GitHub Actions

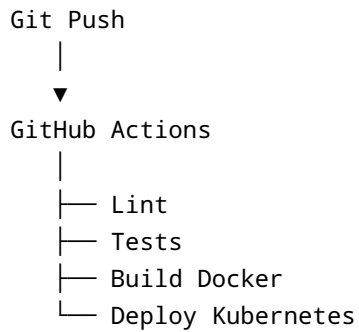
Description

Pipeline CI/CD automatisé GitHub.

Fonctions

- Build automatique
- Exécution des tests
- Vérification qualité
- Déploiement automatique

Workflow CI



5.5 GitLab CI/CD

Description

Pipeline alternatif CI/CD.

Étapes pipeline

Étape	Description
Build	Construction image Docker
Test	Validation application
Deploy	Déploiement cluster

5.6 Jenkins

Description

Serveur d'automatisation DevOps.

Utilisation

- Pipelines personnalisés

- Déploiements avancés
- Automatisation entreprise

Jenkinsfile

Le projet contient un Jenkinsfile permettant l'exécution des pipelines.

5.7 Terraform

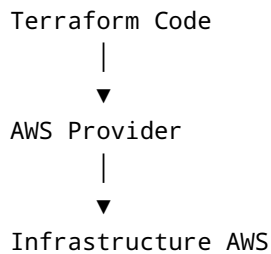
Description

Terraform automatise la création de l'infrastructure AWS.

Ressources AWS prévues

- EC2
- VPC
- Security Groups
- EKS Kubernetes
- IAM

Architecture Terraform



Avantages

- Reproductibilité
 - Versioning infrastructure
 - Déploiement rapide
 - Gestion centralisée
-

5.8 Monitoring Prometheus & Grafana

Prometheus

Rôle

- Collecte des métriques
- Scraping API
- Alerting

Métriques surveillées

Métrique	Description
CPU	Utilisation CPU
RAM	Mémoire
HTTP Requests	Nombre de requêtes
Latency	Temps réponse
Errors	Taux erreurs

Grafana

Rôle

- Visualisation dashboards
- Monitoring temps réel
- Alertes graphiques

Dashboard prévu

```
+-----+
| Requests/sec |
+-----+
| CPU Usage    |
+-----+
| Error Rate   |
+-----+
| API Latency  |
+-----+
```


6. Structure du Projet

```
.
├── app/
│   ├── main.py
│   └── __init__.py
├── tests/
│   ├── test_api.py
│   └── __init__.py
├── k8s/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── ingress.yaml
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── provider.tf
├── monitoring/
│   ├── prometheus.yml
│   └── grafana-dashboard.json
├── .github/
│   └── workflows/
│       └── ci.yml
├── Jenkinsfile
├── .gitlab-ci.yml
├── Dockerfile
├── docker-compose.monitoring.yml
├── requirements.txt
├── pytest.ini
└── README.md
```

7. Exigences Fonctionnelles

7.1 API Chatbot

Le système doit :

- Répondre aux requêtes utilisateurs ;

- Retourner des réponses JSON ;
 - Gérer les erreurs ;
 - Exposer des endpoints de monitoring.
-

7.2 Monitoring

Le système doit :

- Collecter les métriques ;
 - Afficher les dashboards ;
 - Détecter les anomalies ;
 - Envoyer des alertes.
-

7.3 Scalabilité

Le système doit :

- Monter automatiquement en charge ;
 - Gérer plusieurs pods ;
 - Supporter un trafic élevé.
-

8. Exigences Non Fonctionnelles

Exigence	Description
Performance	Temps réponse inférieur à 300ms
Disponibilité	99.9% uptime
Sécurité	HTTPS + Secrets Kubernetes
Scalabilité	Autoscaling horizontal
Maintenabilité	Infrastructure modulaire
Portabilité	Docker compatible multi-cloud

9. Sécurité

Mesures de sécurité

- Gestion des secrets
- Variables d'environnement
- Isolation Docker
- IAM AWS

- HTTPS
- Contrôle d'accès Kubernetes

Bonnes pratiques

- Rotation des clés
- RBAC Kubernetes
- Monitoring sécurité
- Logs centralisés

10. Processus CI/CD Détaillé

Étapes complètes

1. Developer Push
2. Trigger Pipeline
3. Lint Code
4. Execute Tests
5. Build Docker Image
6. Push Registry
7. Deploy Kubernetes
8. Monitoring Validation
9. Production Ready

11. Stratégie de Tests

Tests unitaires

Validation des fonctions backend.

Tests API

Validation des endpoints FastAPI.

Tests CI

Validation pipeline DevOps.

Outils utilisés

Outil	Usage
Pytest	Tests Python

Outil	Usage
Flake8	Linting
GitHub Actions	Validation automatique

12. Déploiement

Déploiement local

Étapes

```
docker build -t chatbot-ai .  
docker run -p 8000:8000 chatbot-ai
```

Déploiement Kubernetes

```
kubectl apply -f k8s/
```

Déploiement Terraform AWS

```
terraform init  
terraform apply
```

13. Gestion des Logs

Logs applicatifs

- Logs FastAPI
- Logs erreurs
- Logs accès API

Logs infrastructure

- Logs Kubernetes
 - Logs CI/CD
 - Logs monitoring
-

14. Monitoring et Alerting

Alertes prévues

Alerte	Condition
CPU élevé	> 80%
Mémoire élevée	> 85%
API indisponible	Healthcheck failed
Latence élevée	> 1 seconde

15. Livrables du Projet

Livrables techniques

- Code source
- Dockerfile
- Pipelines CI/CD
- Scripts Terraform
- Manifests Kubernetes
- Dashboards Grafana

Documentation

- README
- Documentation API
- Documentation déploiement
- Documentation monitoring
- Cahier des charges

16. Planning Prévisionnel

Phase	Durée
Analyse	1 semaine
Développement API	2 semaines
Dockerisation	3 jours
Kubernetes	1 semaine
CI/CD	1 semaine
Monitoring	4 jours

Phase	Durée
Terraform AWS	1 semaine
Tests & Validation	1 semaine

17. Risques du Projet

Risque	Solution
Surcharge Kubernetes	Autoscaling
Panne AWS	Multi-zone
Pipeline échoue	Rollback automatique
Faibles sécurité	Monitoring sécurité

18. Évolutions Futures

Fonctionnalités futures

- Intégration IA avancée
- Chatbot NLP
- Multi-cloud
- Observabilité avancée
- GitOps ArgoCD
- Service Mesh Istio
- ELK Stack
- Sécurité Zero Trust

19. Conclusion

Le projet ChatOps AI représente une plateforme DevOps complète, moderne et scalable permettant de déployer un chatbot IA dans un environnement cloud-native sécurisé.

Grâce à l'automatisation CI/CD, Kubernetes, Docker, Terraform et aux outils de monitoring, la solution garantit :

- Haute disponibilité ;
- Scalabilité ;
- Observabilité ;
- Automatisation ;
- Fiabilité ;
- Rapidité de déploiement.

Ce projet constitue une base solide pour une architecture DevOps professionnelle orientée production et intelligence artificielle.